

TaskFlow

Multi-threaded Job Queue Simulator

VITyarthi — Build Your Own Project
Course: Programming in Java

Submitted by: Aklesh
VIT Bhopal

2. Introduction

TaskFlow is a Java-based simulation of a multi-threaded job scheduling engine. It models a problem common to real-world backend systems: given a stream of incoming work items with different priorities, how should a system queue, schedule, and execute them concurrently while remaining reliable under failure? This project was built to demonstrate practical understanding of core and advanced Java concepts covered in the course — object-oriented design, the Collections framework, exception handling, and the `java.util.concurrent` package — by applying them to a working, testable system rather than isolated exercises.

The system accepts a batch of tasks (each with a name, priority, and estimated duration), processes them concurrently using a fixed-size thread pool, automatically retries failed tasks once, logs every lifecycle event to a file, and generates a performance report summarizing how the run went.

3. Problem Statement

Naively processing tasks one at a time, or without prioritization, wastes system resources and delays high-priority work. Systems such as CI/CD pipelines, background job processors, and print spoolers all face the same underlying challenge: many units of work must be queued fairly, processed concurrently within a bounded resource budget, and recovered gracefully when individual units fail. TaskFlow simulates such a scheduler in a single JVM instance, providing a minimal but functionally complete reference implementation.

4. Functional Requirements

4.1 Task Submission & Priority Queueing

- Tasks are validated on submission (non-blank name, priority ≥ 1 , positive duration).
- Valid tasks are placed into a thread-safe `PriorityBlockingQueue`.
- Tasks are ordered by priority (1 = highest); equal-priority tasks are FIFO-ordered by submission time.
- Invalid submissions raise a custom `InvalidTaskException` rather than corrupting queue state.

4.2 Multi-threaded Task Executor

- A configurable fixed-size thread pool (`java.util.concurrent.ExecutorService`) processes tasks concurrently.
- Each worker thread continuously pulls the highest-priority available task and executes it.
- A task that fails is automatically retried once before being marked permanently failed.
- Thread-safe counters (`AtomicInteger`) track completed, failed, and retried tasks across all worker threads.
- Graceful shutdown drains in-flight work and releases all threads cleanly.

4.3 Logging & Performance Report Generator

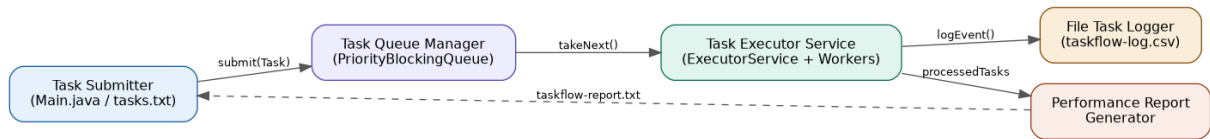
- Every task lifecycle transition (started, completed, failed, retrying) is written to a CSV log file with a timestamp and the worker thread name.
- After a run completes, a performance report is generated: total tasks, completion/failure counts, failure rate, average wait time, average execution time, throughput, and a full per-task breakdown.

5. Non-Functional Requirements

Requirement	How it's addressed
Performance	Thread pool size is configurable to match available CPU cores; queue operations run in $O(\log n)$.
Reliability	Failed tasks are automatically retried once before being marked failed; no task is silently dropped.
Scalability	Pool size and other tunables are centralized in ConfigLoader rather than hardcoded across the codebase.
Maintainability	Code is split into focused packages (model, queue, executor, logger, report, exception, util) with a single responsibility.
Error handling	Custom checked exceptions (InvalidTaskException, TaskExecutionException) separate validation failures from execution failures.
Logging / monitoring	Every state transition is logged to a file with timestamp and thread name for post-run analysis.

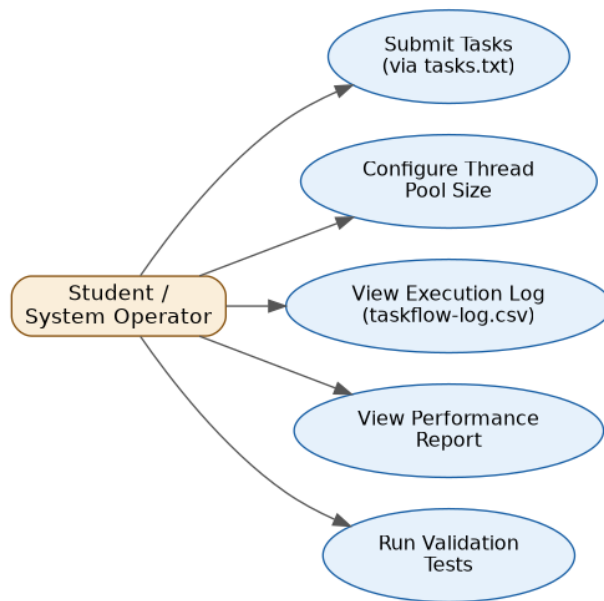
6. System Architecture

The system is organized around a simple pipeline: a task submitter feeds validated tasks into a thread-safe priority queue; a pool of worker threads pulls from that queue and executes tasks concurrently; every event is logged to file; and once processing completes, the recorded results are handed to the report generator.

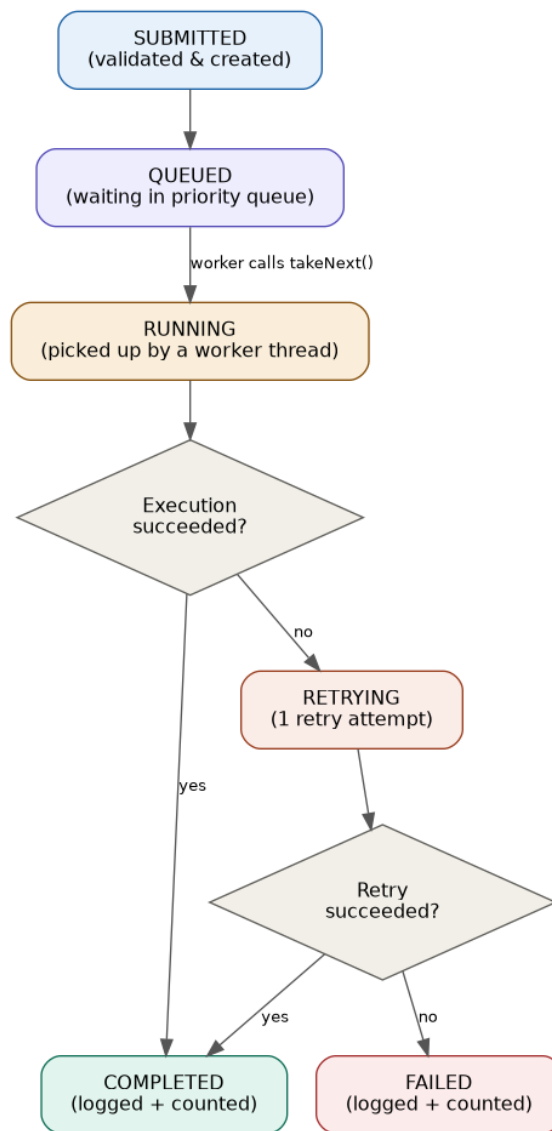


7. Design Diagrams

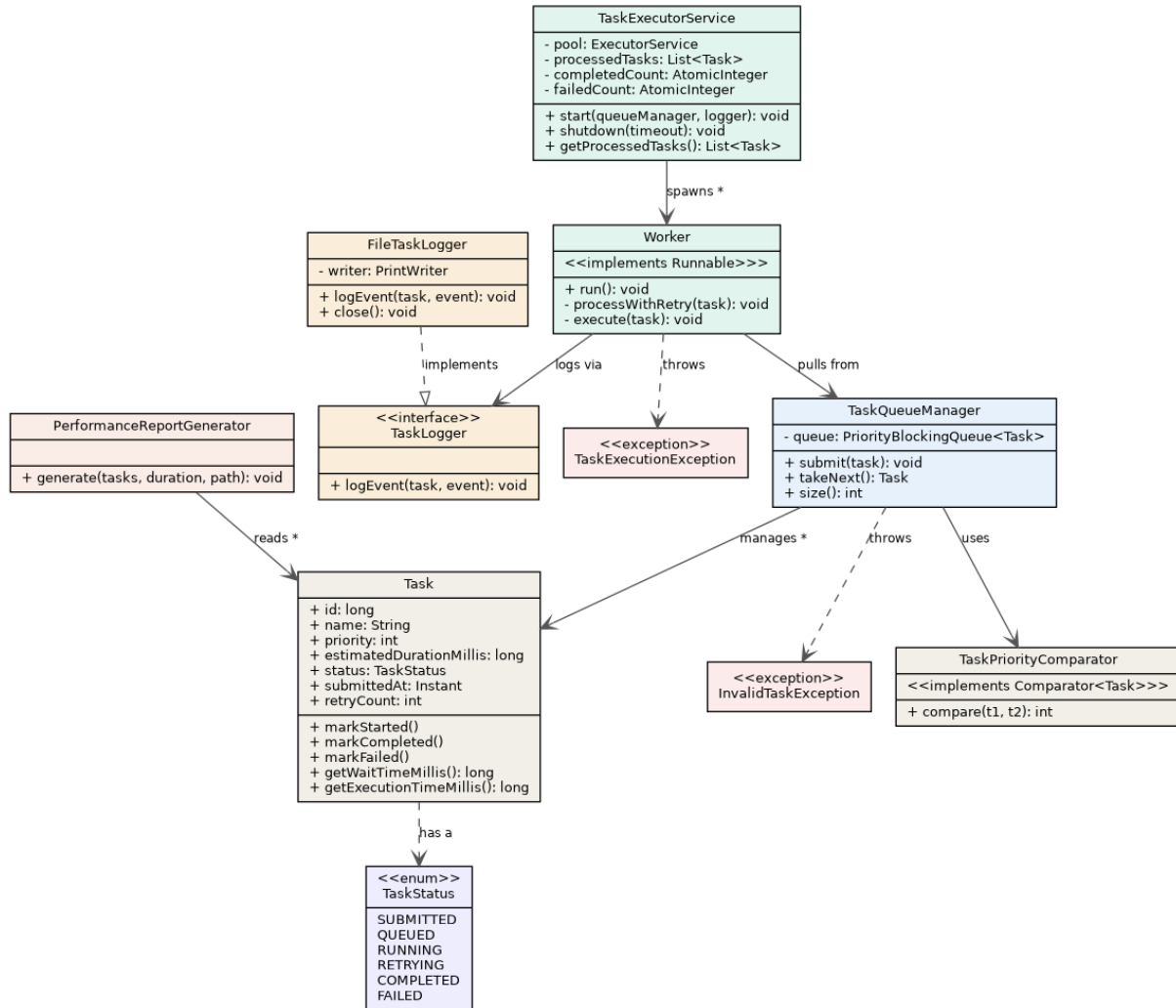
7.1 Use Case Diagram



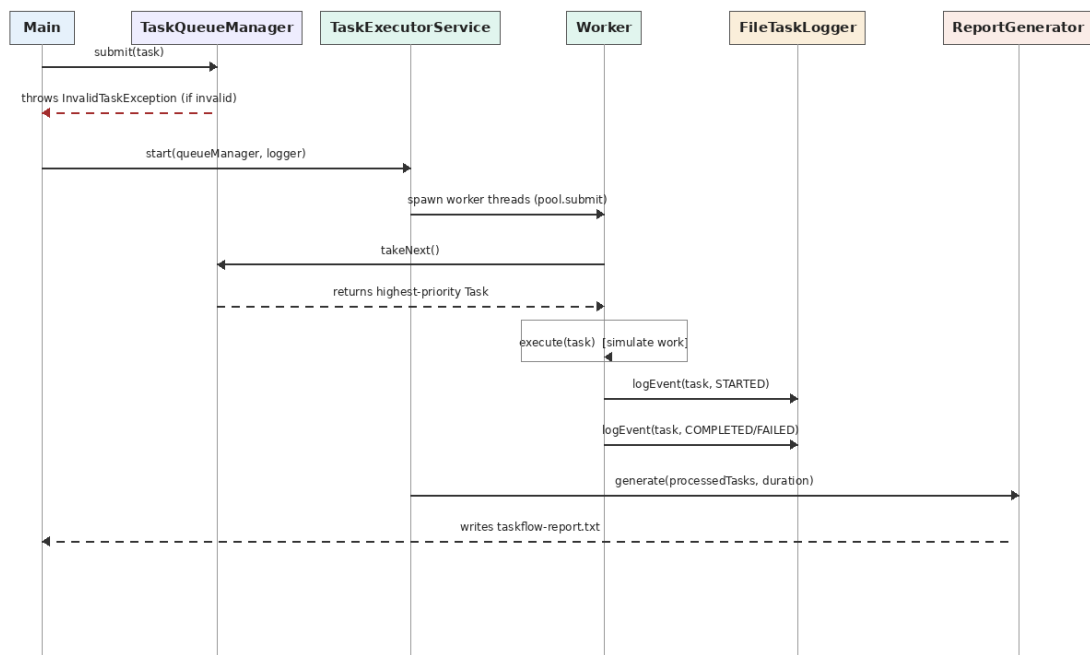
7.2 Workflow / Task Lifecycle Diagram



7.3 Class Diagram



7.4 Sequence Diagram



Note on storage design: TaskFlow uses file-based logging (CSV) rather than a database, so no ER diagram or schema design applies to this project.

8. Design Decisions & Rationale

Why `PriorityBlockingQueue` instead of a manually synchronized structure?

It is a built-in, well-tested thread-safe collection that natively supports blocking `take()` calls, which lets worker threads wait efficiently for new work instead of busy-polling.

Why a custom `Comparator` instead of making `Task` implement `Comparable`?

Decoupling ordering logic from the `Task` class keeps `Task` focused on representing data, and makes it easy to swap in a different ordering strategy later without touching the model class.

Why retry exactly once rather than a configurable retry count?

For a simulation of this scope, a single retry is enough to demonstrate meaningful failure-recovery behavior without adding unnecessary complexity; `MAX_RETRIES` is still a named constant so it can be changed in one place if needed.

Why a `TaskLogger` interface rather than calling `FileTaskLogger` directly?

This follows the dependency inversion principle: the `Worker` class depends on an abstraction, not a concrete file-writing implementation. It also made incremental development possible — `ConsoleTaskLogger` was used to test Module 2 before Module 3's file logger existed.

Why `CopyOnWriteArrayList` for tracking processed tasks?

Multiple worker threads append to this list concurrently, but it's read only once at the end (by the report generator). `CopyOnWriteArrayList` is optimized for exactly this read-rarely/write-often-by-many-threads pattern and avoids the need for manual synchronization.

Why separate `InvalidTaskException` and `TaskExecutionException`?

They represent distinct failure categories — one is a submission-time validation error, the other is a runtime execution failure — and calling code needs to handle them differently (rejecting a bad submission vs. retrying a failed execution).

9. Implementation Details

The codebase is organized into 7 packages and 14 Java files, summarized below:

Package	Classes	Responsibility
model	<code>Task</code> , <code>TaskStatus</code> , <code>TaskPriorityComparator</code>	Core data representation and ordering logic
queue	<code>TaskQueueManager</code>	Thread-safe submission and retrieval
executor	<code>TaskExecutorService</code> , <code>Worker</code>	Thread pool management and task execution with retry
logger	<code>TaskLogger</code> , <code>ConsoleTaskLogger</code> , <code>FileTaskLogger</code>	Pluggable event logging
report	<code>PerformanceReportGenerator</code>	Post-run statistics and reporting
exception	<code>InvalidTaskException</code> , <code>TaskExecutionException</code>	Domain-specific error handling
util	<code>ConfigLoader</code>	Centralized configuration defaults

Concurrency is handled entirely through java.util.concurrent primitives: ExecutorService manages the worker thread pool, PriorityBlockingQueue handles safe concurrent access to pending tasks, AtomicInteger tracks counts without locks, and CopyOnWriteArrayList safely collects results from multiple threads. No manual wait()/notify() or synchronized blocks were needed, which keeps the concurrency model simple to reason about and less prone to deadlocks.

10. Screenshots / Results

10.1 Console Output (Main.java)

```
Loaded 10 tasks (0 rejected). Pool size: 4
Processing complete:
Completed: 8
Failed: 2
Retried: 3
Log written to taskflow-log.csv
Report written to taskflow-report.txt
```

10.2 Generated Performance Report

```
=== TaskFlow Performance Report ===
Generated at: 2026-09-11T20:35:17.365774603Z

Total tasks processed : 10
Completed : 9
Failed : 1
Failure rate : 10.00%

Average wait time : 582.80 ms
Average execution time : 1019.60 ms
Throughput : 3.32 tasks/sec

Run duration : 3013 ms

=== Per-task Breakdown ===
TaskID,Name,Priority,Status,WaitMs,ExecMs,Retries
2,Send email,1,COMPLETED,30,433,0
10,Notify subscribers,1,COMPLETED,34,527,0
4,Cleanup temp files,1,COMPLETED,33,629,0
7,Refresh cache,1,COMPLETED,32,640,1
5,Sync inventory,2,COMPLETED,563,900,0
1,Generate report,3,COMPLETED,673,1201,0
6,Compress logs,3,COMPLETED,1464,700,0
3,Backup database,2,COMPLETED,463,1801,0
8,Validate orders,2,COMPLETED,662,2202,1
9,Archive old records,3,FAILED,1874,1163,1
```

10.3 Sample Log Output (taskflow-log.csv, excerpt)

```
timestamp,taskId,taskName,thread,event
2026-09-11T20:35:14.334593997Z,2,Send email,pool-1-thread-2,STARTED on pool-1-thread-2
2026-09-11T20:35:14.365375594Z,10,Notify subscribers,pool-1-thread-4,STARTED on
pool-1-thread-4
2026-09-11T20:35:14.365539840Z,7,Refresh cache,pool-1-thread-3,STARTED on pool-1-thread-3
2026-09-11T20:35:14.365628403Z,4,Cleanup temp files,pool-1-thread-1,STARTED on pool-1-thread-1
2026-09-11T20:35:14.675573998Z,7,Refresh cache,pool-1-thread-3,FAILED (attempt 1): Simulated
random failure for task 7
2026-09-11T20:35:14.675963462Z,7,Refresh cache,pool-1-thread-3,RETRYING
2026-09-11T20:35:14.766174230Z,2,Send email,pool-1-thread-2,COMPLETED
2026-09-11T20:35:14.766626754Z,3,Backup database,pool-1-thread-2,STARTED on pool-1-thread-2
2026-09-11T20:35:14.865788836Z,10,Notify subscribers,pool-1-thread-4,COMPLETED
2026-09-11T20:35:14.866129013Z,5,Sync inventory,pool-1-thread-4,STARTED on pool-1-thread-4
2026-09-11T20:35:14.965940062Z,4,Cleanup temp files,pool-1-thread-1,COMPLETED
```

11. Testing Approach

Testing was done in two layers. First, each module had a standalone smoke-test class (Module1Test, Module2Test, Module3Test) written and run during development to verify that module's behavior in isolation before integrating it with the rest of the system. Second, an automated validation suite (ValidationTests.java) was built with 20 explicit assertions covering:

- Priority ordering and FIFO tie-breaking in the queue
- Rejection of invalid task submissions (blank name, non-positive priority/duration)
- Queue size and empty-state reporting as tasks are added and removed
- Task timing calculations (wait time, execution time) at each lifecycle stage
- A full concurrent run through the real thread pool, verifying every task reaches a final state and completed+failed counts reconcile with the total submitted

All 20 checks pass on every run:

Running TaskFlow validation tests...

```
[PASS] Priority ordering: highest priority (1) comes out first
[PASS] Priority ordering: FIFO tiebreak between equal priorities
[PASS] Priority ordering: mid priority (2) comes third
[PASS] Priority ordering: low priority (3) comes last
[PASS] Validation: blank task name is rejected
[PASS] Validation: priority < 1 is rejected
[PASS] Validation: non-positive duration is rejected
[PASS] Queue: reports empty before any submission
[PASS] Queue: size reflects number of submitted tasks
[PASS] Queue: size decreases after takeNext()
[PASS] Queue: reports empty after draining all tasks
[PASS] Timing: wait time is -1 before task starts
[PASS] Timing: execution time is -1 before task completes
[PASS] Timing: status becomes RUNNING after markStarted()
[PASS] Timing: wait time is non-negative once started
[PASS] Timing: status becomes COMPLETED after markCompleted()
[PASS] Timing: execution time is positive after completion
[2026-09-11T20:35:52.932871735Z] Task#13 (Integration job 3) -> STARTED on pool-1-thread-2
[2026-09-11T20:35:52.936746819Z] Task#16 (Integration job 6) -> STARTED on pool-1-thread-1
[2026-09-11T20:35:52.936809840Z] Task#19 (Integration job 9) -> STARTED on pool-1-thread-3
[2026-09-11T20:35:53.014739036Z] Task#13 (Integration job 3) -> COMPLETED
[2026-09-11T20:35:53.015021846Z] Task#11 (Integration job 1) -> STARTED on pool-1-thread-2
[2026-09-11T20:35:53.015246402Z] Task#19 (Integration job 9) -> COMPLETED
[2026-09-11T20:35:53.015429993Z] Task#14 (Integration job 4) -> STARTED on pool-1-thread-3
[2026-09-11T20:35:53.015984805Z] Task#16 (Integration job 6) -> COMPLETED
[2026-09-11T20:35:53.016571472Z] Task#17 (Integration job 7) -> STARTED on pool-1-thread-1
```

```
[2026-09-11T20:35:53.065400658Z] Task#11 (Integration job 1) -> COMPLETED
[2026-09-11T20:35:53.065628467Z] Task#14 (Integration job 4) -> COMPLETED
[2026-09-11T20:35:53.065903186Z] Task#20 (Integration job 10) -> STARTED on pool-1-thread-3
[2026-09-11T20:35:53.066671398Z] Task#12 (Integration job 2) -> STARTED on pool-1-thread-2
[2026-09-11T20:35:53.067156628Z] Task#17 (Integration job 7) -> COMPLETED
[2026-09-11T20:35:53.067720366Z] Task#15 (Integration job 5) -> STARTED on pool-1-thread-1
[2026-09-11T20:35:53.116218916Z] Task#20 (Integration job 10) -> COMPLETED
[2026-09-11T20:35:53.116894629Z] Task#18 (Integration job 8) -> STARTED on pool-1-thread-3
[2026-09-11T20:35:53.117992619Z] Task#15 (Integration job 5) -> COMPLETED
[2026-09-11T20:35:53.118413416Z] Task#12 (Integration job 2) -> COMPLETED
[2026-09-11T20:35:53.167499198Z] Task#18 (Integration job 8) -> COMPLETED
[PASS] Concurrency: every submitted task reaches a final state
[PASS] Concurrency: completed + failed counts add up to total tasks
[PASS] Concurrency: no task is left in a non-final status
```

--- Summary ---

Passed: 20

Failed: 0

12. Challenges Faced

- **Coordinating shutdown cleanly:** worker threads block on `queue.take()`, so a plain `shutdown()` call wasn't enough to stop them — `shutdownNow()` was needed to interrupt the blocking call and let each worker's loop exit.
- **Avoiding race conditions in shared counters:** multiple worker threads update `completed/failed/retried` counts simultaneously; `AtomicInteger` was used instead of plain `int` fields to avoid lost updates without needing explicit locks.
- **Deciding where to draw the retry boundary:** retry logic needed to live close to the execution call so a second failure could be distinguished from the first, without duplicating the logging and counting logic — this shaped the structure of `Worker.processWithRetry()`.
- **Testing concurrent code deterministically:** because thread scheduling and simulated random failures are inherently non-deterministic, the integration test in `ValidationTests` checks invariants that must hold regardless of execution order (e.g. `completed + failed == total`) rather than exact outcomes.

13. Learnings & Key Takeaways

- Practical experience with `java.util.concurrent` beyond basic `Thread/Runnable` — using `ExecutorService`, `PriorityBlockingQueue`, `AtomicInteger`, and `CopyOnWriteArrayList` together in one system.
- Reinforced that thread-safety is a property of shared state and how it's accessed, not just a checkbox — every shared field in this project (counters, the task list, the queue) needed a deliberate thread-safe choice.
- Interfaces (`TaskLogger`) make incremental, testable development possible — a simpler console implementation let later modules be tested before the final file-based one existed.
- Writing invariant-based tests (rather than exact-output tests) is necessary once concurrency is involved, since exact timing and ordering can't be guaranteed run-to-run.

14. Future Enhancements

- Persist tasks and results to a database (e.g. SQLite/MySQL via JDBC) instead of flat files, enabling historical queries across multiple runs.
- Make retry count and backoff delay configurable per task rather than a fixed constant.
- Add a simple JavaFX or web dashboard to visualize queue depth and throughput in real time.
- Support task dependencies (task B only runs after task A completes) using a DAG-based scheduler.
- Replace the simulated random failure with pluggable real task logic (e.g. actual file processing, HTTP calls) for a more realistic workload.

15. References

- Oracle Java Documentation — `java.util.concurrent` package: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/package-summary.html>
- Oracle Java Documentation — `ExecutorService` interface
- Oracle Java Documentation — `PriorityBlockingQueue` class
- VITyarthi "Build Your Own Project" — General Project Instructions & Submission Guidelines